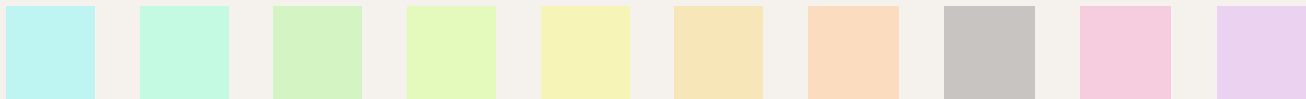


Client Side JavaScript

מבוא לבניית מערכות ממוחשבות מבוססות אינטרנט הרצאה 6 – JavaScript



JS



JavaScript

- JavaScript was invented by Brendan Eich in 1995 and became an ECMA standard in 1997 (European Computer Manufacturers Association).
- JavaScript and JAVA are completely different languages.
- Although there are similarities between JavaScript and Java, including language name, syntax, and respective standard libraries, the two languages are distinct and differ greatly in design.

JavaScript Abilities

1. JavaScript Can Change HTML Content
2. JavaScript Can Change HTML Attribute Values
3. JavaScript Can Change HTML Styles (CSS)
4. JavaScript Can Hide HTML Elements
5. JavaScript Can Show HTML Elements

(1) JavaScript Can Change HTML Content

- One of many JavaScript HTML methods is getElementById().
- The example below "finds" an HTML element (with id="demo"), and changes the element content (innerHTML) to "Hello JavaScript":

```
document.getElementById("demo").innerHTML = "Hello JavaScript";
```

JavaScript accepts both double and single quotes

(2) JavaScript Can Change HTML Attribute Values

- JavaScript changes the value of the src (source) attribute of an `<img>`

```
document.getElementById('myPicture').src='picB.gif'
```

(3) JavaScript Can Change HTML Styles (CSS)

- Changing the style of an HTML element, is a variant of changing an HTML attribute

```
document.getElementById("demo").style.fontSize = "35px";
```

(4) JavaScript Can Hide HTML Elements

- Hiding HTML elements can be done by changing the display style

```
document.getElementById("demo").style.display = "none";
```


(5) JavaScript Can Show HTML Elements

- Showing hidden HTML elements can also be done by changing the display style

```
document.getElementById("demo").style.display = "block";
```



Inserting JavaScript

Inserting JavaScript

- There are 3 options to insert JavaScript:
 1. Internal – in `<script>` tag in `<head>`
 2. Internal – in `<script>` tag in `<body>`
 3. External – in a separate JS file

Internal JavaScript- the <script> Tag

- In HTML, JavaScript code is inserted between <script> and </script> tags.

```
<script>  
    document.getElementById("demo").innerHTML = "My First JavaScript";  
</script>
```



JavaScript in <head> or <body>

- You can place any number of scripts in an HTML document.
- Scripts can be placed in the <body>, or in the <head> section of an HTML page, or in both.
- Usually, in **<head> we will insert scripts that are critical for the page to load**
- Usually, in **<body> we will insert other scripts at the bottom of the code** (before </body>).
 - why?
 - Placing scripts at the bottom of the <body> element improves the display speed, because script interpretation slows down the display.

JavaScript In <head>

```
<!DOCTYPE html>
<html>
<head>
<script>
function myFunction() {
  document.getElementById("P1").innerHTML = "Paragraph changed.";
}
</script>
</head>
<body>
<h1>A Web Page</h1>
<p id="P1">A Paragraph</p>
<button type="button" onclick="myFunction()">Push da button</button>
</body>
</html>
```

The function is invoked
(called) when a button is
clicked



JavaScript in <body>

```
<!DOCTYPE html>
<html>
<body>

<h1>A Web Page</h1>
<p id="demo">A Paragraph</p>
<button type="button" onclick="myFunction()">Try it</button>

<script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>

</body>
</html>
```

The function is invoked
(called) when a button is
clicked



External JavaScript

- Create external JS file: FileName.js
- To use an external script, put the name of the script file in the src (source) attribute of a <script> tag

```
<script src="myScript.js"></script>
```

- Place the <script> element connecting to the external JS code into the <head> element



External JavaScript Advantages

- It separates HTML and code
- It makes HTML and JavaScript easier to read and maintain
- Cached JavaScript files (JS stored in the cache memory) can speed up page loads

HTML <script> defer Attribute

- The defer attribute is a Boolean attribute.
- If the defer attribute is set, it specifies that **the script is downloaded in parallel to parsing the page and executed after the page has finished parsing.**
- Note: The defer attribute is only for external scripts (should only be used if the src attribute is present).

```
<script src="myScript.js" defer></script>
```



JavaScript Output

JavaScript Output

JavaScript can "display" data in different ways:

1. Writing into an HTML element, using `innerHTML`.
2. Writing into the HTML output using `document.write()`.
3. Writing into an alert box, using `alert()`.
4. Writing into the browser console, using `console.log()`.

Using innerHTML

- To access an HTML element, JavaScript can use the `document.getElementById("id")` method.
- The `id` attribute defines the HTML element.
- The `innerHTML` property defines the HTML content

```
<!DOCTYPE html>
<html>
<body>

<h1>My First Web Page</h1>
<p>My First Paragraph</p>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = Date();
</script>

</body>
</html>
```



Using document.write()

- For testing purposes, it is convenient to use document.write()
- Using document.write() after an HTML document is loaded, will **overwrite all existing HTML**:
- The document.write() method should only be used for testing.

```
<!DOCTYPE html>
<html>
  <head>
    <title>document.write</title>
  </head>
  <body>
    <h1>My First Web Page</h1>
    <p>My first paragraph.</p>

    <button type="button" onclick=myFunction()>Try it</button>
    <script>
      function myFunction(){
        document.write('test')
      }
    </script>
  </body>
</html>
```



Using `window.alert()` or simply `alert()`

- You can use an alert box to display data to the **USER!**

```
<!DOCTYPE html>
<html>
<body>

<h1>My First Web Page</h1>
<p>My first paragraph.</p>

<script>
    alert("Hi there!");
</script>

</body>
</html>
```



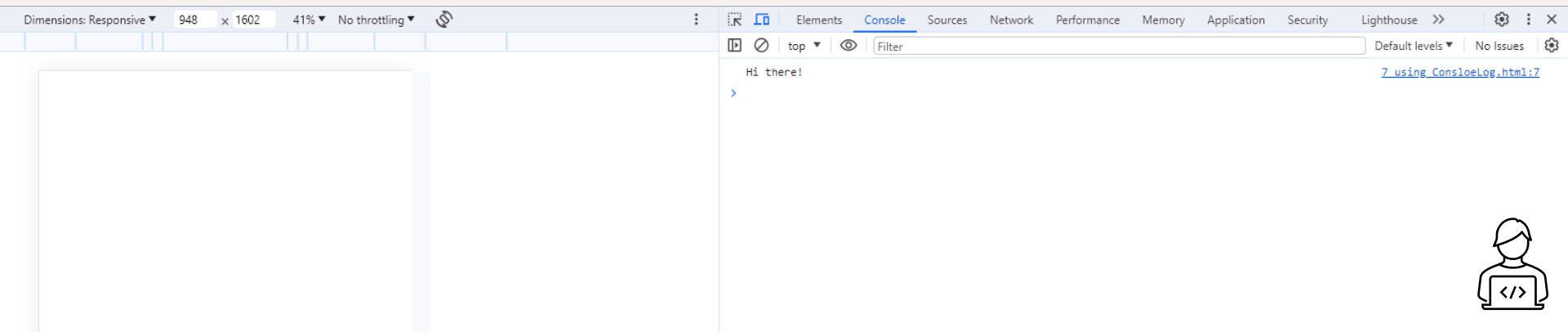
Using `console.log()`

- For debugging purposes, you can call the `console.log()` method in the browser to display data.

```
<!DOCTYPE html>
<html>
<body>

<script>
console.log("Hi there!");
</script>

</body>
</html>
```



JavaScript-General

JavaScript Statements

- Every software code is a list of instructions that need to be executed by the computer.
- A JavaScript program is a list of programming statements.

```
var x, y, z;      // Statement 1
x = 5;            // Statement 2
y = 6;            // Statement 3
z = x + y;        // Statement 4
```

JavaScript Statements

- JavaScript statements are composed of:
 - I. Values
 - II. Operators
 - III. Logical expressions
 - IV. Keywords
 - V. Comments

Semicolons

- Semicolons separate JavaScript statements
- Add a semicolon at the end of each executable statement



Code Blocks

- JavaScript statements can be grouped together in code blocks, inside **curly brackets {...}**.
- The purpose of code blocks is to define statements to be executed together.
- For example- JavaScript functions:

```
function myFunction() {  
    document.getElementById("demo1").innerHTML = "Hello Dolly!";  
    document.getElementById("demo2").innerHTML = "How are you?";  
}
```

JavaScript Keywords (full list)

Keyword	Description
break	Terminates a switch or a loop
continue	Jumps out of a loop and starts at the top
debugger	Stops the execution of JavaScript, and calls (if available) the debugging function
do ... While	Executes a block of statements, and repeats the block, while a condition is true
for	Marks a block of statements to be executed, as long as a condition is true
function	Declares a function
if ... else	Marks a block of statements to be executed, depending on a condition
return	Exits a function
switch	Marks a block of statements to be executed, depending on different cases
try ... catch	Implements error handling to a block of statements
var	Declares a variable



Syntax

Syntax

- JavaScript syntax is the set of rules, defining how JavaScript programs are constructed

```
var x, y, z;           // Declare Variables  
x = 5; y = 6;          // Assign Values  
z = x + y;             // Compute Values
```

JavaScript Values

- The JavaScript syntax defines two types of values:
 1. Fixed values (Literals)
 2. Variable values (Variables)

Literals

- The two most important syntax rules for fixed values are:
 1. Numbers are written with or without decimals

10.50

1001

2. Strings are text, written within double or single quotes

"John Doe"

'John Doe'

JavaScript Variables

- Variables are used to store data values.
- JavaScript uses the **var** keyword to declare variables.
- An equal sign is used to assign values to variables.

```
var x;
```

```
x = 6;
```

JavaScript Comments

- Code after double slashes `//` or between `/*` and `*/` is treated as a comment.

```
var x = 5;    // I will be executed
```

```
/* var x = 6;    I will NOT be executed*/
```


Single Line Comments

- Single line comments start with `//`.

```
// Change heading:  
document.getElementById("myH").innerHTML = "My First Page";  
  
// Change paragraph:  
document.getElementById("myP").innerHTML = "My first paragraph.";
```

```
var x = 5;      // Declare x, give it the value of 5  
var y = x + 2;  // Declare y, give it the value of x + 2
```

Multi-line Comments

- Multi-line comments start with `/*` and end with `*/`.
- Any text between `/*` and `*/` will be ignored by JavaScript.
 - Or just mark the lines you want to insert to a comment and press `ctrl+/*`

```
/*  
The code below will change  
the heading with id = "myH"  
and the paragraph with id = "myP"  
in my web page:  
*/  
document.getElementById("myH").innerHTML = "My First Page";  
document.getElementById("myP").innerHTML = "My first paragraph.";
```



Variables

JavaScript Variables

- JavaScript variables are containers for storing data values.
- In this example, x, y, and z, are variables, declared with the var keyword:

```
var x = 5;  
var y = 6;  
var z = x + y;
```

Declaring Variables: var, let and const

- **var** keyword defines a JavaScript variable.

ES2015 introduced two important new JavaScript keywords:

- **let** keyword **defines a variable in a block scope** and can't be accessed or redeclared outside the block.
- **const** keyword defines a variable that **cannot be reassigned** and must be declared with a value assigned.

Declaring Variables: var, let and const

```
{let x=2;}  
console.log(x);|
```

✖ Uncaught ReferenceError: x is not defined
at [myScript.js:8:13](#)

```
{var x=2;}  
console.log(x);|
```

2

Variables Scope

Global Scope

- Variables declared Globally (outside any function) have Global Scope.

Function Scope \ local scope

- Variables declared Locally (inside a function) have Function Scope.

Block Scope

- Variables declared inside a block {} cannot be accessed from outside the block.
 - Variables declared with the var keyword cannot have Block Scope.
 - Variables declared with the let keyword can have Block Scope.

Let VS Var VS Const

	Scope	Redeclare	Reassign
var	No	Yes	Yes
let	Yes	No	Yes
const	Yes	No	No

Value = undefined

- In many programming languages, you can declare a variable without assigning a value.
- This is useful when:
 - The value needs to be calculated later
 - It will be provided by user input
- Such a variable will have the value undefined (except for const, which must be initialized).

```
let carName;  
console.log(carName); // Output: undefined
```

Naming Variables: JavaScript Identifiers

- JS variables must be identified with unique names called **identifiers**.
- Can be short (like x and y) or descriptive (age, totalVolume).
- General rules for constructing variables names:
 1. Can contain letters, digits, underscores, and dollar signs (\$).
 2. can't begin with a digit.
 3. Case sensitive (y and Y are different variables).
 4. Can't contain spaces.
 5. Reserved words (like JavaScript keywords) cannot be used as names.



Data Types

JavaScript Variables Data Types

1. Text values – strings (single or double quotes)
2. Numbers (with or without decimal point)
3. Booleans (True\False)
4. Arrays ([])
5. Objects ({})
6. Undefined (empty)



Data Types- strings

Strings

- A string (or a text string) is a series of characters like "JavaScript".
- Strings are written with quotes. You can use single or double quotes:

```
var carName1 = "Volvo XC60";    // Using double quotes  
var carName2 = 'Volvo XC60';    // Using single quotes
```

- You can use quotes inside a string, as long as they don't match the quotes surrounding the string:

```
var answer1 = "It's alright";    // Single quote inside double quotes  
var answer3 = 'He's called "John"'; // Double quotes inside single quotes
```

Strings

- Concatenate strings:
 - `'a'+'b'='ab'`
- String length:
 - `myString.length;`
- Slice strings:
 - Using `slice(x,y)`
 - `var name='Itzik';`
 - `name.slice(0,1);`
 - `l`
 - `name.slice(4,5);`
 - `k`

Strings- exercise 1

Ask the user to insert a text, store it in a variable, and alert the user with the number of characters he inserted, and the number of characters he have left out of 140 available characters.

- Hint- use prompt to ask for input
 - `Var userAnswer=propmp("insert text");`
- Hint 2- use alert to inform the user
- Use `.length` to check strings length.



Strings- exercise 2

- After informing the user with the number of characters he have left (out of 140), if the user inserted a longer string, cut it and alert the user with the first 140 characters of his input.
 - Hint- use `.slice(x,y)`





Data Types- Numbers

JavaScript Numbers

- JavaScript has only one type of numbers.
- Numbers can be written with, or without decimals:

```
var x1 = 34.00;    // Written with decimals  
var x2 = 34;       // Written without decimals
```

- Extra large or extra small numbers can be written with scientific (exponential) notation:

```
var y = 123e5;     // 12300000  
var z = 123e-5;    // 0.00123
```

JavaScript Arithmetic Operators

Operator	Description	Example
+	Addition	1+1=2
-	Subtraction	1-1=0
*	Multiplication	2*3=6
**	Exponentiation	3**2=9
/	Division	9/3=3
%	Modulus (Division Remainder)	9%3=0 ; 8%4=0 ; 8%5=3 ; 4%3=1
++	Increment in 1	variable++
--	Decrement in 1	variable--

The Assignment Operator

- In JavaScript, the equal sign (=) is an "assignment" operator, not an "equal to" operator.

The "equal to" operator is written like == or === in JavaScript.

== compares values

=== compares values and datatype

```
let a = 5;
let b = "5";

console.log(a == b);
// true → values are equal (type is ignored)
console.log(a === b);
// false → values are equal, but types differ (number vs string)
```

JavaScript Assignment Operators

Operator	Example	Same As
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y



Data Types- Booleans

JavaScript Booleans

- Booleans can only have two values: true or false.

```
var x = 5;  
var y = 5;  
var z = 6;  
(x == y)      // Returns true  
(x == z)      // Returns false
```

- Booleans are often used in conditional testing.



Data Types- Arrays

JavaScript Arrays

- JavaScript arrays are written with square brackets.
- Array items are separated by commas.
- The following code declares an array called cars, containing three items (car names):

```
var cars = ["Volkswagen", "Volvo", "BMW"];
```

- Array indexes are zero-based, which means the first item is [0], second is [1], and so on (same for strings)

JavaScript Arrays

```
var cars = ["Volkswagen", "Volvo", "BMW"];  
cars.length; //provides the number of items in the array  
>3  
cars.includes("Mazda"); // checks if a values is included in the array  
>False  
cars.includes("Volkswagen");  
>True  
cars.push("Tesla"); // adds an item to the end of the array  
>["Volkswagen", "Volvo", "BMW", "Tesla"]  
cars.unshift("Skoda"); // adds an item to the beginning of the array  
>["Skoda", "Volkswagen", "Volvo", "BMW", "Tesla"]
```



Data Types- Objects

JavaScript Objects

- JavaScript objects are written with curly braces {}.
- Object properties are written as name:value pairs, separated by commas.

```
var person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
```

- The object (person) in the example above has 4 properties: firstName, lastName, age, and eyeColor.



Functions

JavaScript Functions

- A JavaScript function is a block of code designed to perform a particular task.
- A JavaScript function is executed when "something" invokes it (calls it).

JavaScript Function Syntax

- A JavaScript function is defined with the function keyword, followed by a name, followed by parentheses ().

```
function name(parameter1, parameter2, parameter3) {  
    // code to be executed  
}
```

- Function **parameters** are listed inside the parentheses () in the function definition.
- Function **arguments** are the **values** received by the function when it is invoked.
- Inside the function, the arguments (the parameters) behave as local variables.

Function Invocation

- The code inside the function will execute when "something" invokes (calls) the function:
 1. When an event occurs (when a user clicks a button)
 2. When it is invoked (called) from JavaScript code
 3. Automatically (self invoked)

Function Return

- When JavaScript reaches a return statement, the function will stop executing.
- If the function was invoked from a statement, JavaScript will "return" to execute the code after the invoking statement.
- Functions often compute a return value.
- The return value is "returned" back to the "caller"

```
var x = myFunction(4, 3);  
// Function is called, return value will end up in x  
  
function myFunction(a, b) {  
    return a * b;  
// Function returns the product of a and b  
}
```

Functions- exercise

- Go to : <https://stanford.edu/~cpiech/karel/ide.html>
- Change the world to 5X5
- Under references you can find all available functions of the robot
- Write a function that drives the robot to the top right corner.
- Write a function that puts beepers on the diagonal of the board

